

CO4-AT2 – Question 28

Name: R.Gautham Krishna

Register No: 192421128

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Online Advertising Budget Allocation Using Knapsack + Greedy Hybrid

1. Problem Statement

A digital marketing firm has a fixed advertising budget and needs to allocate this budget among multiple advertising campaigns to maximize the expected return on investment (ROI).

Each campaign requires a specific amount of money and provides an expected return. Since the total budget is limited, the company cannot select every campaign.

The objective is to select the most profitable combination of campaigns while ensuring that the total advertising cost does not exceed the available budget.

The problem can be modeled as a 0/1 Knapsack Problem, where:

- Weight = Advertising cost
- Value = Expected revenue/return
- Capacity = Total advertising budget

A hybrid approach combining Greedy and Dynamic Programming is used to improve practical decision-making.

2. Industrial Scenario

Consider a digital marketing company with a total advertising budget of:

Budget = ₹10,000

The company has five advertising campaigns:

Campaign	Advertising Cost	Expected Return
Social Media	₹2,000	₹3,500
Search Ads	₹3,000	₹5,000
Video Ads	₹4,000	₹6,000

Campaign	Advertising Cost	Expected Return
----------	------------------	-----------------

Email Marketing	₹1,000	₹1,500
-----------------	--------	--------

Influencer Ads	₹5,000	₹7,000
----------------	--------	--------

The company must select campaigns such that:

Total Cost $\leq$ ₹10,000

while maximizing:

Total Expected Return

3. Objectives

The objectives of this project are:

1. To model advertising budget allocation as a Knapsack optimization problem.
2. To maximize expected advertising revenue.
3. To ensure that the total campaign cost does not exceed the available budget.
4. To use a Greedy approach for quick campaign selection.
5. To use Dynamic Programming to obtain an optimal solution.
6. To combine both approaches for practical real-time decision-making.
7. To analyze algorithm efficiency.

4. Constraints Considered

The following constraints are considered:

1. Maximum Advertising Budget

The company has a fixed budget.

Maximum Budget = ₹10,000

2. Campaign Cost

Each campaign requires a specific advertising investment.

For example:

Social Media = ₹2,000

3. Campaign Return

Each campaign generates an expected return.

For example:

Social Media Return = ₹3,500

4. Budget Constraint

The total cost of selected campaigns must satisfy:

Total Cost $\leq$ ₹10,000

5. Campaign Selection

In the 0/1 Knapsack model, each campaign can be:

- Selected once
- Not selected

A campaign cannot be partially selected.

6. Performance Uncertainty

Expected campaign returns may change because of:

- Customer behaviour
- Market trends
- Competition
- Advertisement performance
- Seasonal demand

5. Algorithm Used

Hybrid Knapsack + Greedy + Dynamic Programming

The proposed solution combines two approaches.

Greedy Approach

The campaigns are initially ranked according to their return-to-cost ratio.

Return-to-Cost Ratio = Expected Return / Advertising Cost

Campaigns with higher ratios are considered first.

Greedy is useful because it provides a solution quickly.

Dynamic Programming

Dynamic Programming is then used to evaluate possible combinations and find the optimal allocation under the budget constraint.

The DP table is represented as:

$DP[i][b]$

Where:

- i = number of campaigns considered
- b = available budget
- $DP[i][b]$ = maximum expected return

6. Algorithm Steps

Step 1

Initialize the advertising budget.

Budget = ₹10,000

Step 2

Store the cost and expected return of each campaign.

Step 3

Calculate the return-to-cost ratio for each campaign.

Step 4

Sort campaigns based on their ratio using the Greedy approach.

Step 5

Use the Greedy result as an initial feasible solution.

Step 6

Create a Dynamic Programming table.

Step 7

For every campaign and budget value:

- Include the campaign if possible.
- Exclude the campaign.
- Compare both returns.
- Store the maximum return.

Step 8

Find the maximum expected return from the DP table.

Step 9

Trace back the DP table to identify the selected campaigns.

7. Python Source Code

```
main.py Visualize Run
1 campaigns = [
2     ("Social Media", 2000, 3500),
3     ("Search Ads", 3000, 5000),
4     ("Video Ads", 4000, 6000),
5     ("Email Marketing", 1000, 1500),
6     ("Influencer Ads", 5000, 7000)
7 ]
8 budget = 10000
9 campaigns_sorted = sorted(
10     campaigns,
11     key=lambda x: x[2] / x[1],
12     reverse=True
13 )
14 print("Greedy Ranking:")
15 for name, cost, value in campaigns_sorted:
16     print(name, "Ratio =", round(value / cost, 2))
17 n = len(campaigns)
18 dp = [[0] * (budget + 1) for _ in range(n + 1)]
19 for i in range(1, n + 1):
20     name, cost, value = campaigns[i - 1]
21     for b in range(budget + 1):
22         dp[i][b] = dp[i - 1][b]
23         if cost <= b:
24             dp[i][b] = max(
25                 dp[i][b],
26                 value + dp[i - 1][b - cost]
27             )
28 print("\nMaximum Expected Return =", dp[n][budget])
```

8. Correct Output

```
Output
Greedy Ranking:
Social Media Ratio = 1.75
Search Ads Ratio = 1.67
Video Ads Ratio = 1.5
Email Marketing Ratio = 1.5
Influencer Ads Ratio = 1.4

Maximum Expected Return = 16000
```

One optimal selection is:

Campaign	Cost	Return
Social Media	₹2,000	₹3,500
Search Ads	₹3,000	₹5,000
Video Ads	₹4,000	₹6,000
Email Marketing	₹1,000	₹1,500
Total	₹10,000	₹16,000

Note: For these sample values, the total return of all five campaigns is ₹16,000, exactly matching the ₹10,000 budget. Therefore, the correct optimal result is ₹16,000, not ₹15,500.

9. Analysis

The algorithm analyzes the cost and expected return of every advertising campaign.

The system considers:

- Advertising budget
- Campaign cost
- Expected return
- Return-to-cost ratio
- Campaign selection
- Budget utilization

The Greedy method quickly identifies campaigns with high return per rupee.

For example:

Social Media:

$$3500 / 2000 = 1.75$$

Therefore, Social Media has a high return-to-cost ratio.

However, Greedy alone does not always guarantee the optimal solution for a 0/1 Knapsack problem.

Dynamic Programming evaluates different campaign combinations and guarantees the optimal solution for the given integer-budget model.

10. Real-Time Decision Making

Advertising performance can change continuously in real-world applications.

For example:

Morning → Search Ads perform well

Afternoon → Social Media performs better

Evening → Video Ads receive more engagement

A real-time system can update:

- Campaign cost
- Expected revenue
- Click-through rate
- Conversion rate
- Customer engagement
- Available budget

The Greedy algorithm can provide a quick decision when immediate action is required.

The Dynamic Programming algorithm can be periodically executed to recalculate the optimal allocation.

Therefore, the hybrid approach is suitable for systems where both speed and optimization are important.

11. Revenue Maximization

The objective is to maximize:

Total Revenue = Sum of Expected Returns

while satisfying:

Total Campaign Cost $\leq$ Advertising Budget

For example, if the company spends ₹10,000 and generates ₹16,000 expected return:

Investment = ₹10,000

Expected Return = ₹16,000

Expected net gain:

₹16,000 - ₹10,000 = ₹6,000

Thus, the optimized allocation helps maximize the expected revenue within the available advertising budget.

12. Hybrid Strategy

The hybrid strategy works in two stages.

Stage 1 – Greedy

Rank campaigns based on:

Return / Cost

This quickly identifies attractive campaigns.

Stage 2 – Dynamic Programming

Use Dynamic Programming to examine combinations of campaigns and determine the maximum possible return.

Therefore:

Greedy → Fast initial solution

Dynamic Programming → Optimal solution

This combination provides a balance between computational speed and solution quality.

13. Computational Complexity

Let:

- n = Number of advertising campaigns
- B = Advertising budget

Greedy Complexity

Sorting campaigns based on return-to-cost ratio:

Time Complexity – $O(n \log n)$

Dynamic Programming Complexity

The DP table contains $n \times B$ states.

Time Complexity – $O(n \times B)$

Space Complexity – $O(n \times B)$

Using a one-dimensional DP array, space can be optimized to:

Space Complexity – $O(B)$

Overall Hybrid Complexity

Time Complexity – $O(n \log n + nB)$

Space Complexity – $O(nB)$

14. Implementation Challenges

1. Changing Campaign Performance

Campaign performance may change during the day.

Solution:

Update expected returns using real-time campaign data.

2. Large Number of Campaigns

A company may have hundreds or thousands of campaigns.

Solution:

Use Greedy filtering to remove low-performing campaigns before applying DP.

3. Large Advertising Budget

A large budget can create a large DP table.

Solution:

Use budget scaling or a one-dimensional DP implementation.

4. Uncertain Returns

Expected revenue is not always guaranteed.

Solution:

Use historical performance and predictive analytics to estimate returns.

5. Real-Time Requirements

Dynamic Programming may take time when the number of campaigns and budget values are very large.

Solution:

Use Greedy for immediate decisions and run DP periodically for optimization.

15. Result Interpretation

The Knapsack + Greedy Hybrid approach successfully determines how the advertising budget can be allocated among different campaigns.

For the given example:

Total Budget = ₹10,000

Maximum Expected Return = ₹16,000

The selected campaigns use the entire available budget while maximizing the expected return.

The solution demonstrates that budget allocation can be treated as an optimization problem where each campaign has a cost and expected benefit.

16. Business Applications

This solution can be applied to:

- Google advertising
- Social media advertising
- YouTube campaigns
- Email marketing
- Influencer marketing
- E-commerce promotions
- Mobile advertising
- Digital marketing platforms

The system can help companies decide where to spend their advertising money to obtain maximum expected revenue.

17. Advantages

- Maximizes expected advertising revenue.
- Respects the fixed budget constraint.
- Greedy provides fast decisions.
- Dynamic Programming provides an optimal solution.
- Can handle multiple campaigns.
- Can be adapted for real-time decision-making.
- Helps improve advertising ROI.
- Reduces inefficient budget allocation.

18. Conclusion

The Online Advertising Budget Allocation problem can be effectively modeled as a 0/1 Knapsack optimization problem, where campaign cost represents the weight and expected revenue represents the value.

The proposed Greedy + Dynamic Programming hybrid approach provides both fast decision-making and optimal budget allocation. Greedy ranking can quickly identify promising campaigns, while Dynamic Programming evaluates possible combinations to maximize the expected return.

For the given scenario, the system utilizes the ₹10,000 advertising budget and achieves an expected maximum return of ₹16,000.

Overall Time Complexity – $O(n \log n + nB)$ | Space Complexity – $O(nB)$

This approach can help digital marketing companies maximize revenue, improve ROI, and make efficient real-time advertising decisions.